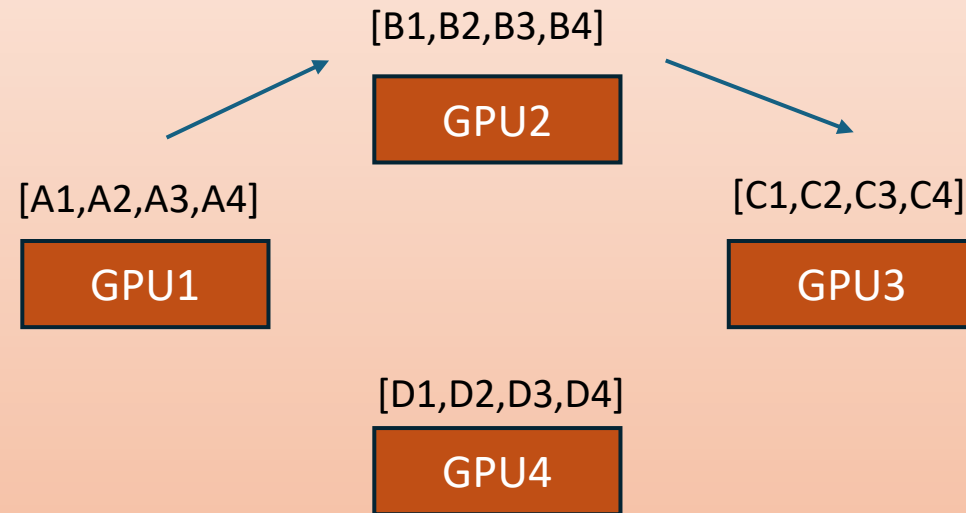
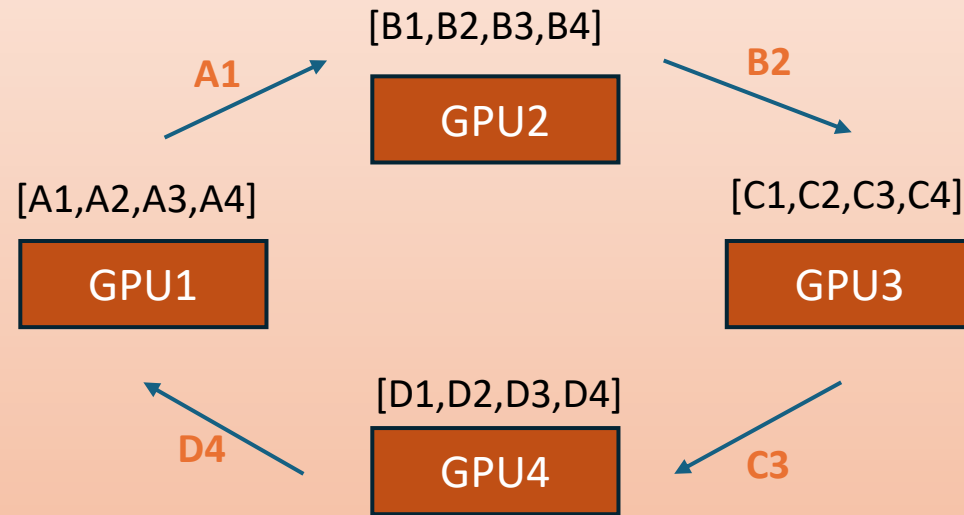


Ring All-Reduce example



1. Initial State: Each GPU splits its gradient into 4 chunks (1 per GPU).

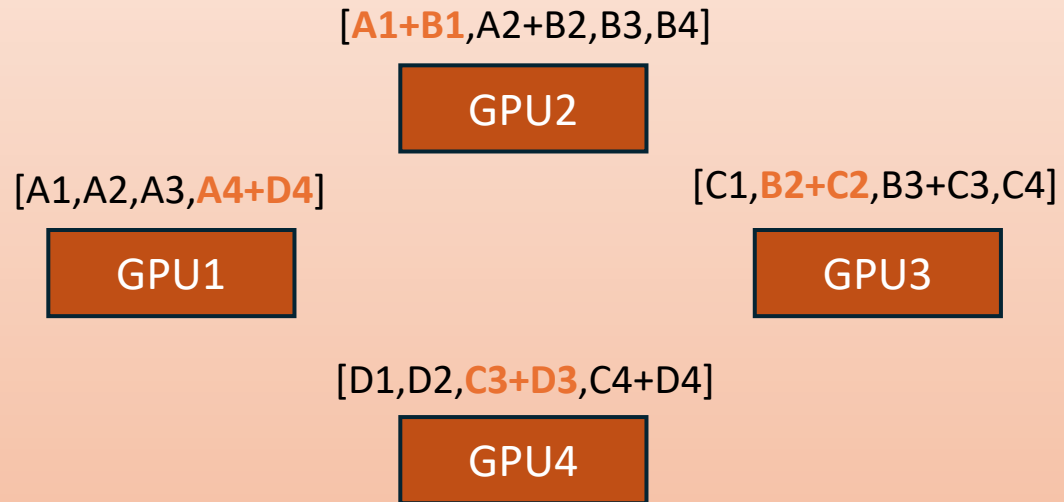
Ring All-Reduce example



"Share" Step

2. Ring Communication (Reduce – Scatter) phase : All GPUs send chunks to their neighbors. The chunks are added (reduced).

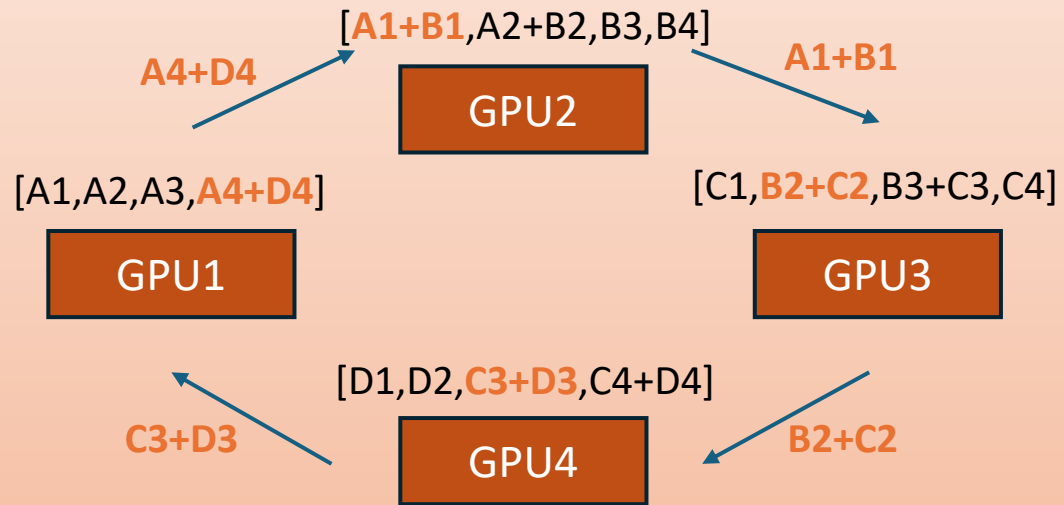
Ring All-Reduce example



“Reduce” Step

2. Ring Communication (Reduce – Scatter)
phase : All GPUs send chunks to their neighbors. The chunks are added (reduced).

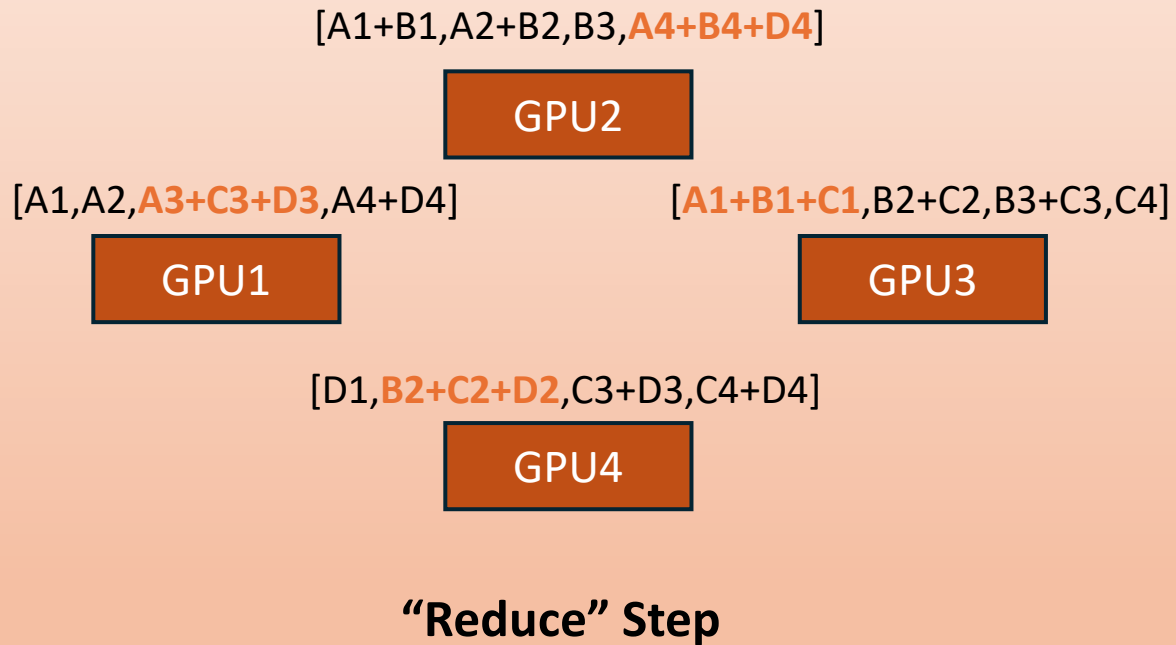
Ring All-Reduce example



"Share" Step

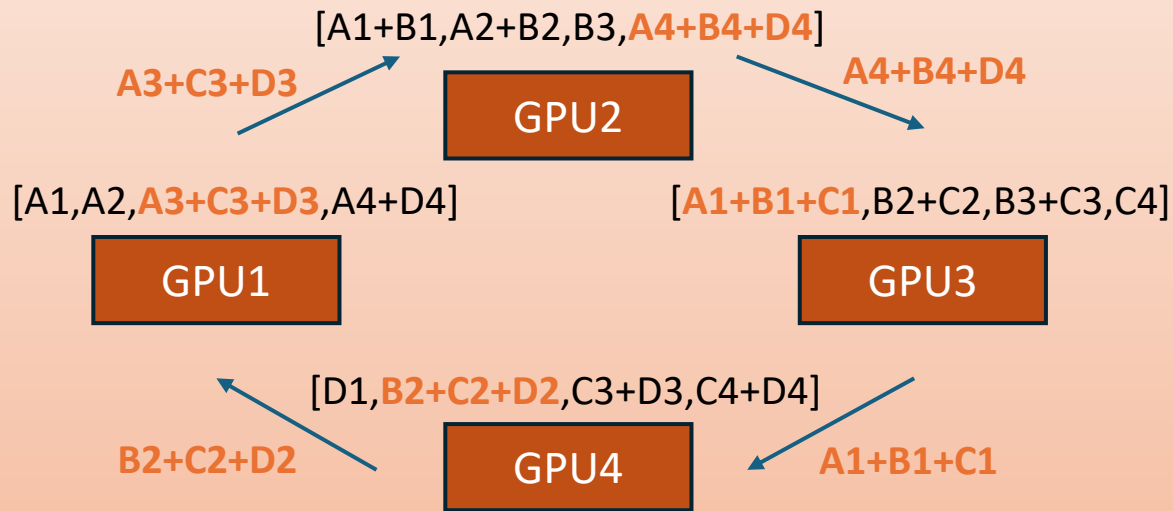
2. Ring Communication (Reduce – Scatter) phase : All GPUs send chunks to their neighbors. The chunks are added (reduced).

Ring All-Reduce example



2. Ring Communication (Reduce – Scatter) phase : All GPUs send chunks to their neighbors. The chunks are added (reduced).

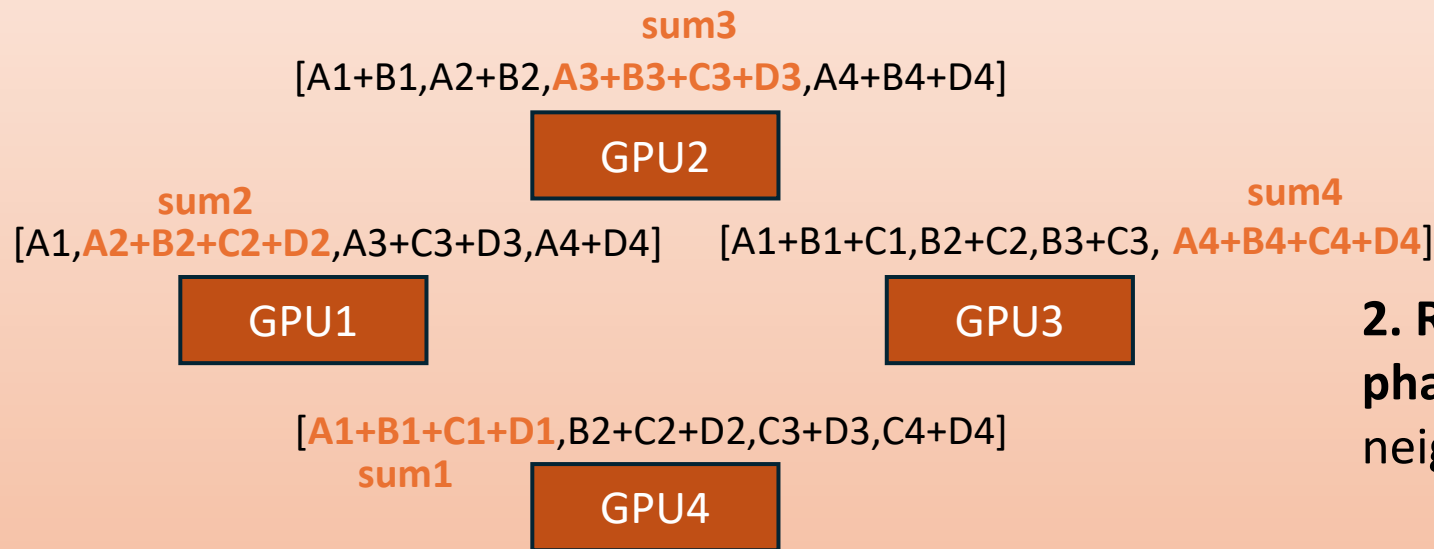
Ring All-Reduce example



“Share” Step

2. Ring Communication (Reduce – Scatter) phase : All GPUs send chunks to their neighbors. The chunks are added (reduced).

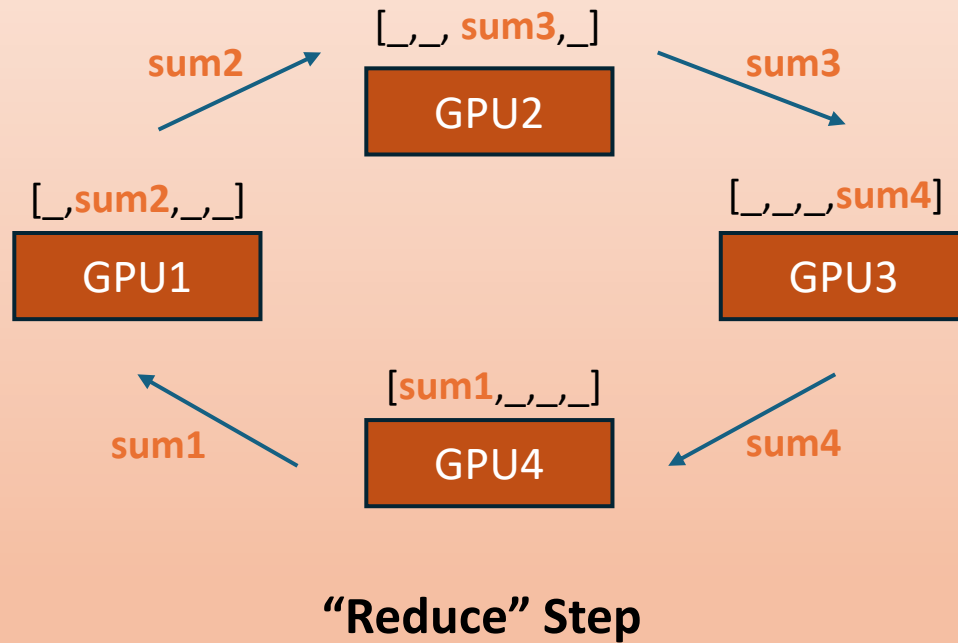
Ring All-Reduce example



“Reduce” Step

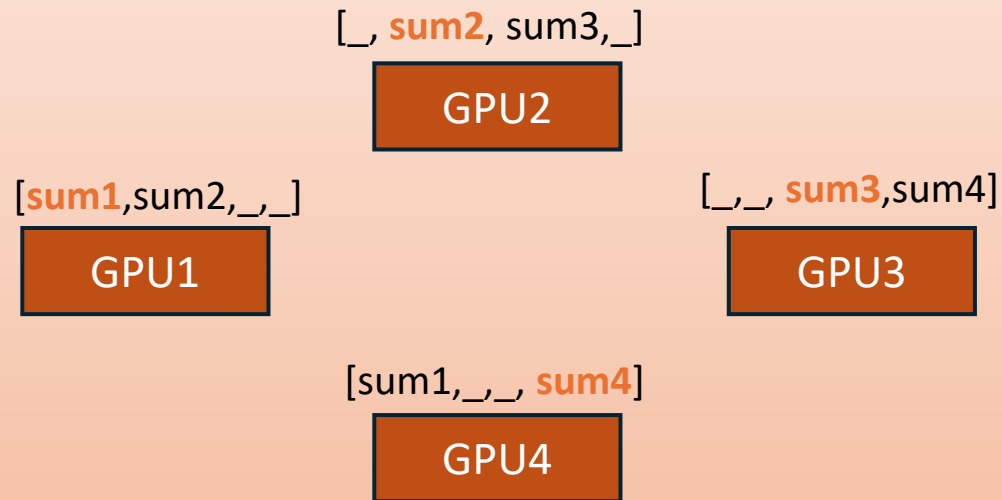
2. Ring Communication (Reduce – Scatter)
phase: All GPUs send chunks to their neighbors. The chunks are added (reduced).

Ring All-Reduce example



3. All-gather (broadcast) phase: each GPU sends its fully summed chunk to its neighbor

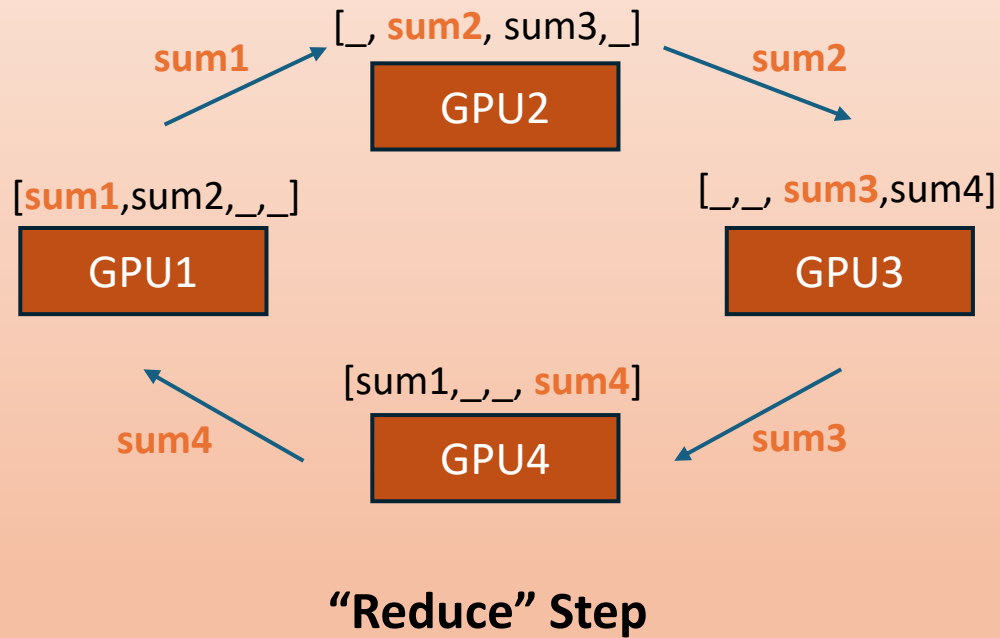
Ring All-Reduce example



“Reduce” Step

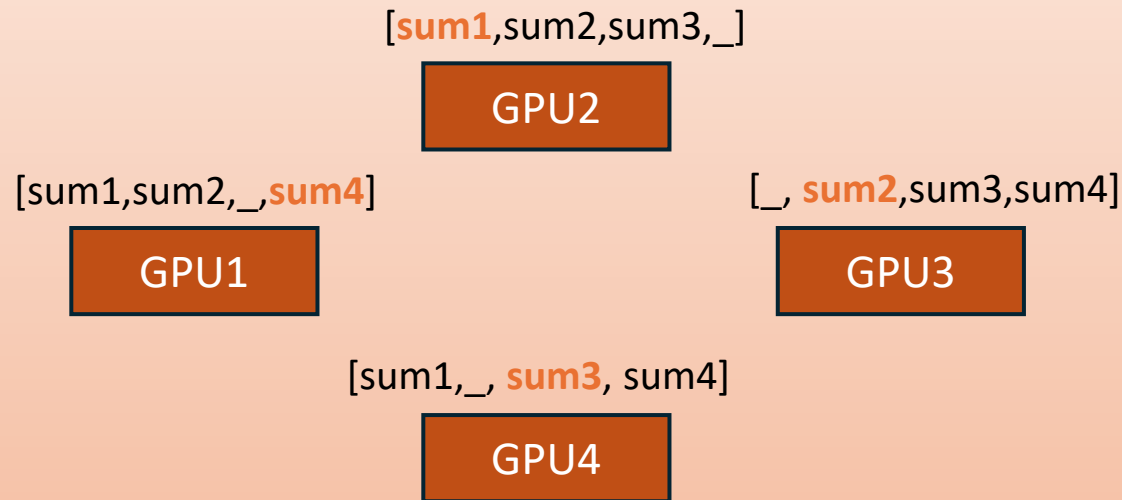
3. All-gather (broadcast) phase: each GPU sends its fully summed chunk to its neighbor

Ring All-Reduce example



3. All-gather (broadcast) phase: each GPU sends its fully summed chunk to its neighbor

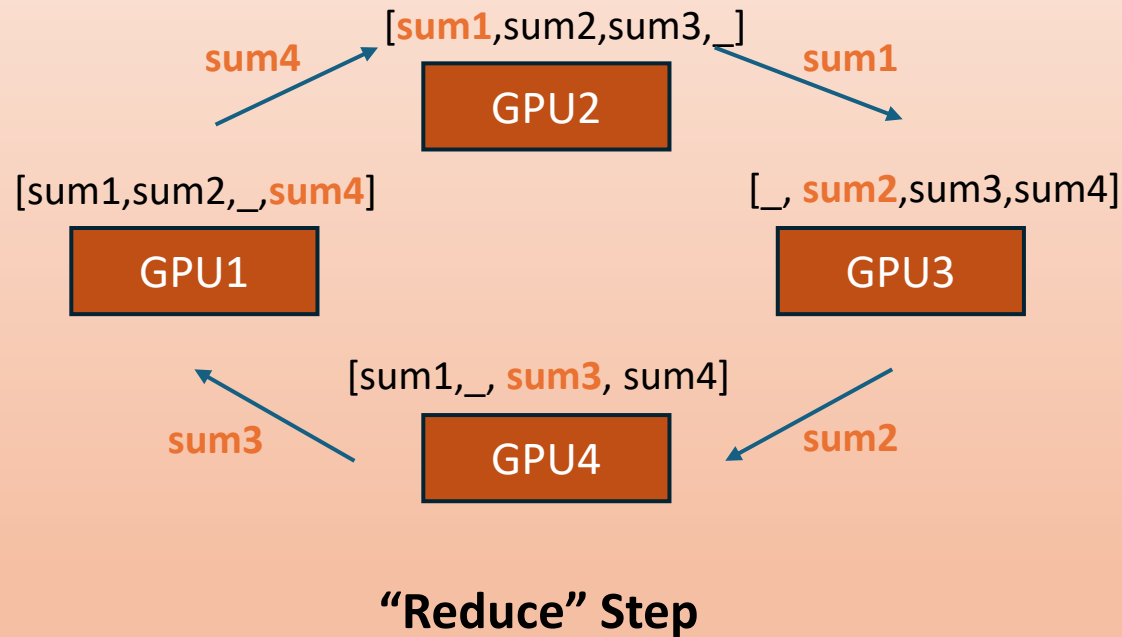
Ring All-Reduce example



“Reduce” Step

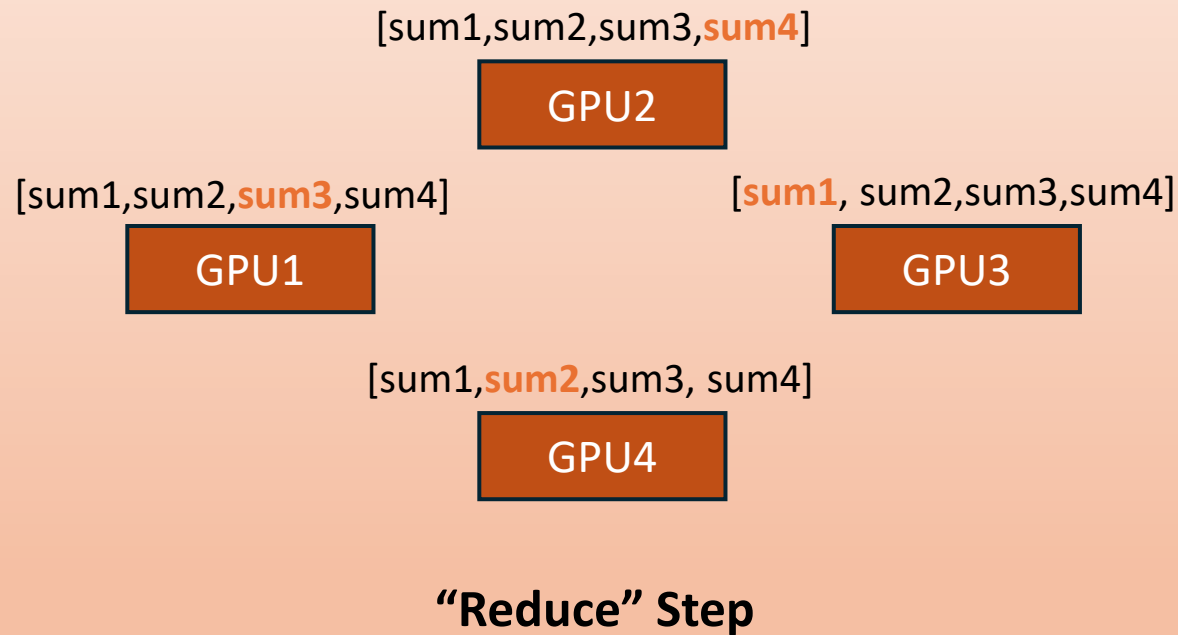
3. All-gather (broadcast) phase: each GPU sends its fully summed chunk to its neighbor

Ring All-Reduce example



3. All-gather (broadcast) phase: each GPU sends its fully summed chunk to its neighbor

Ring All-Reduce example



3. All-gather (broadcast) phase: each GPU sends its fully summed chunk to its neighbor

Communication Optimization

BF16 is a 16-bit floating-point format that sacrifices precision for a wide dynamic range, making it stable for training neural networks. FP16 is a 16-bit format that offers higher precision than BF16 but has a narrower range, which can cause overflow or underflow issues during training.

ResNet-50 is a popular model for image recognition whose key innovation was a "shortcut" that allows the core signal, or meaningful information, to skip over layers. These shortcuts prevent the signal from getting weak and lost in deep networks, providing a direct path that makes them much easier to train accurately.

How big are the gradients?

- Gradient size = $P \text{ params} \times b \text{ bytes/param}$
- FP32 = 4 bytes; BF16/FP16 = 2 bytes

Ring all-reduce traffic per device:

- $2 \times P \times b \times (N-1)/N$

Example (ResNet-50 $\approx$ 25M params)

- FP32 gradients $\approx$ 100 MB
- $N = 8$ GPUs $\rightarrow \approx$ 175 MB traffic per device per step

Fabrics matter

- NVLink $\sim$ 300 GB/s $\rightarrow \sim$ 0.6 ms for 175 MB (idealized)
- 100 Gbps Ethernet/InfiniBand $\approx$ 12.5 GB/s $\rightarrow \sim$ 14 ms (idealized)

Make it faster

- Overlap comm with compute via gradient bucketing
- Choose bucket sizes to avoid many tiny messages
- Use BF16/FP16 gradients to halve traffic (with proper scaling)
- For small tensors, tree-based reductions can beat ring due to lower latency

Distributed Systems Concerns in Practice

Fabrics and topology:

- Intra-node: PCIe, NVLink
- Inter-node: 25/100/200/400 Gbps Ethernet/InfiniBand; latencies 2–10× higher

Collectives vs parameter server:

- Collectives (NCCL/Gloo/MPI): bandwidth-efficient, symmetric
- PS: flexible, supports async, but can bottleneck at servers

Orchestration & launch:

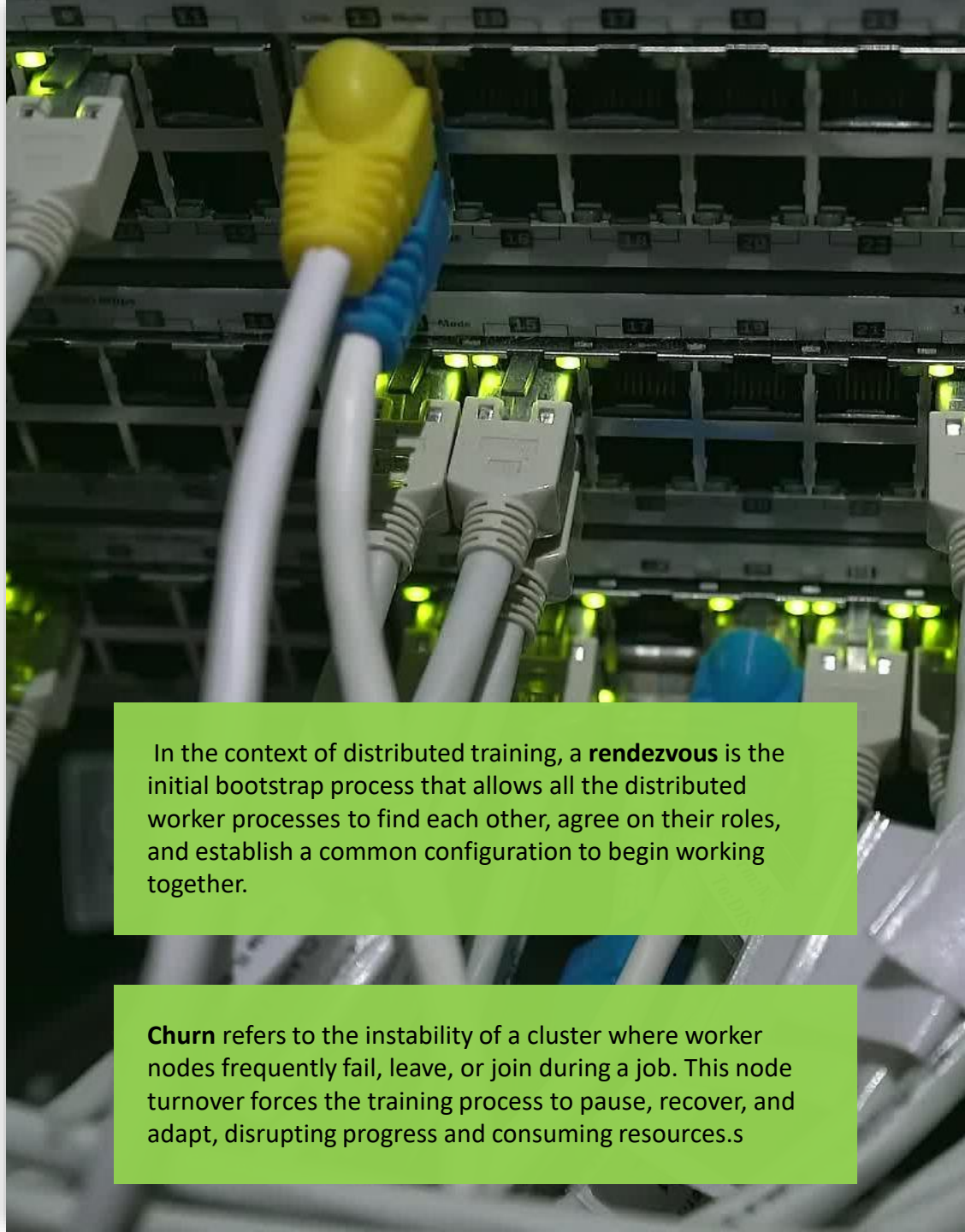
- SLURM/Kubernetes; ranks, world size, rendezvous; topology-aware placement

Reproducibility:

- Deterministic sharding and seeding; avoid nondeterministic kernels

Elasticity:

- Handle node churn; reshard data and adjust LR safely



In the context of distributed training, a **rendezvous** is the initial bootstrap process that allows all the distributed worker processes to find each other, agree on their roles, and establish a common configuration to begin working together.

Churn refers to the instability of a cluster where worker nodes frequently fail, leave, or join during a job. This node turnover forces the training process to pause, recover, and adapt, disrupting progress and consuming resources.

Sync vs Async vs Elastic Data Parallel

Synchronous all-reduce:

- Pros: stable convergence; single-node equivalence
- Cons: straggler-sensitive; per-step barrier

Parameter-server (async/stale-sync):

- Pros: more throughput; tolerant to slow clients
- Cons: stale gradients; careful LR tuning; potential accuracy hit

Elastic DP (synchronous with changing world size):

- World size adjusts at runtime; reshard data/checkpoints; adjust LR with effective batch

Guidance: prefer sync for accuracy-critical; use async/elastic when churn is inevitable

Topology-Aware & Hierarchical All-Reduce; Libraries

- Libraries:
 - NCCL (GPU-centric), Gloo (CPU/GPU), MPI (HPC)
- Algorithms:
 - Ring: bandwidth-optimal for large messages
 - Tree: lower latency for smaller messages
 - Reduce-scatter + all-gather: core of sharded training
- Hierarchical all-reduce:
 - Intra-node reduce-scatter (NVLink)
 - Inter-node reduce-scatter (IB/Ethernet)
 - Inter-node all-gather
 - Intra-node all-gather
- Practice: bucket sizes, algorithm selection per tensor size, topology hints